

Node服务器和常见模块

王红元 coderwhy

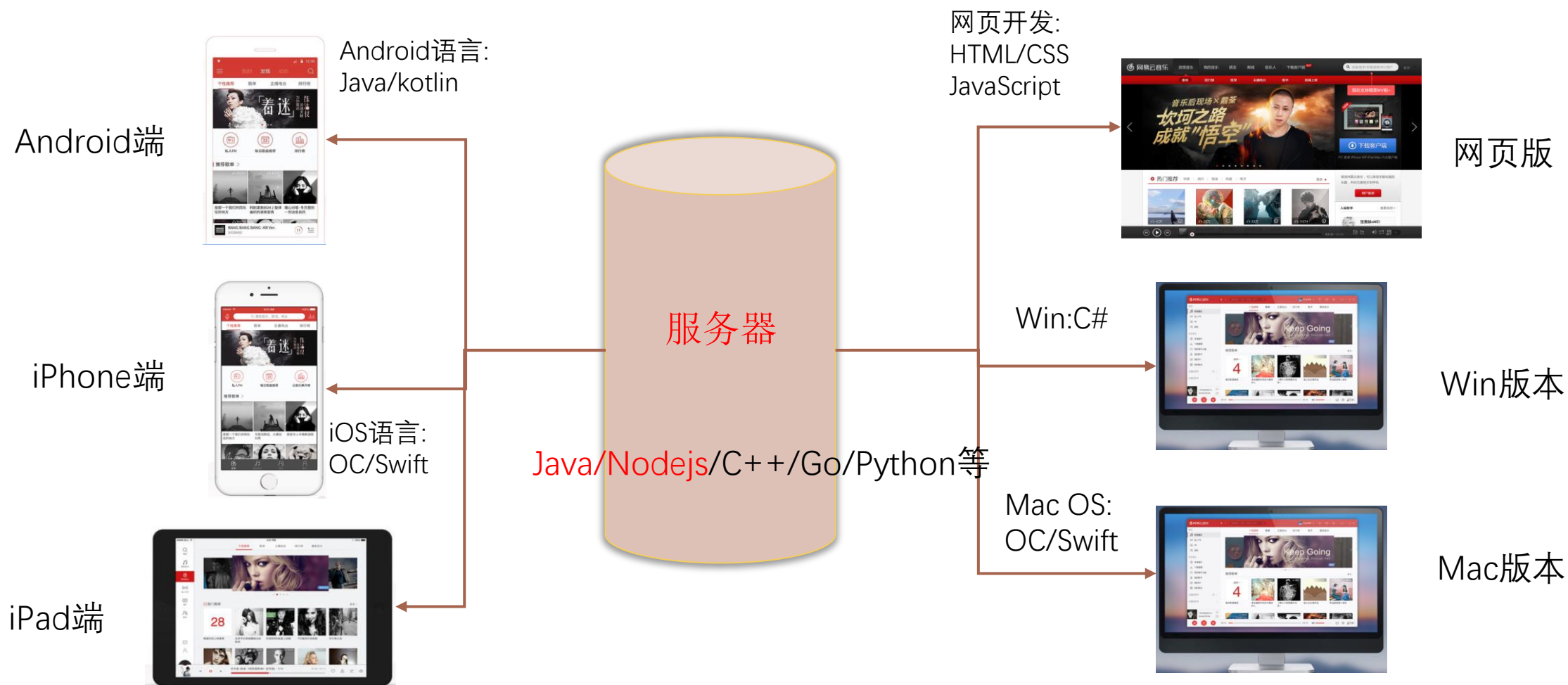
目录

content



- 1 Node服务器开发
- 2 fs模块-文件系统
- 3 event模块-事件处理
- 4 认识二进制和buffer
- 5 Buffer的创建方式
- 6 Buffer的源码解析

服务器开发



Node.js是什么

■ 官方对Node.js的定义:

- Node.js是一个基于V8 JavaScript引擎的JavaScript运行时环境。

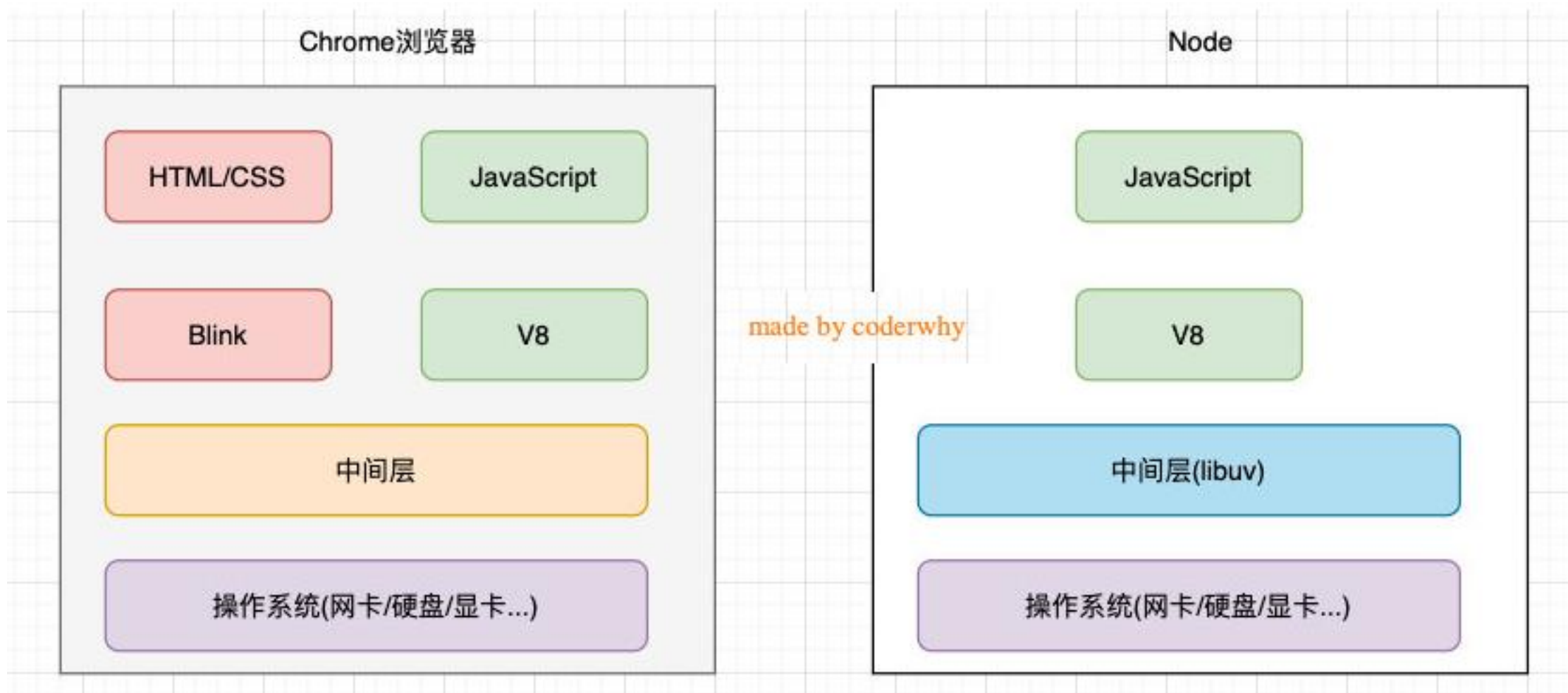
Node.js® is a JavaScript runtime built on Chrome's V8 JavaScript engine.

■ 也就是说Node.js基于V8引擎来执行JavaScript的代码，但是不仅仅只有V8引擎:

- 前面我们知道V8可以嵌入到任何C ++应用程序中，无论是Chrome还是Node.js，事实上都是嵌入了V8引擎来执行JavaScript代码;
- 但是在Chrome浏览器中，还需要解析、渲染HTML、CSS等相关渲染引擎，另外还需要提供支持浏览器操作的API、浏览器自己的事件循环等;
- 另外，在Node.js中我们也需要进行一些额外的操作，比如文件系统读/写、网络IO、加密、压缩解压文件等操作;

浏览器和Node.js架构区别

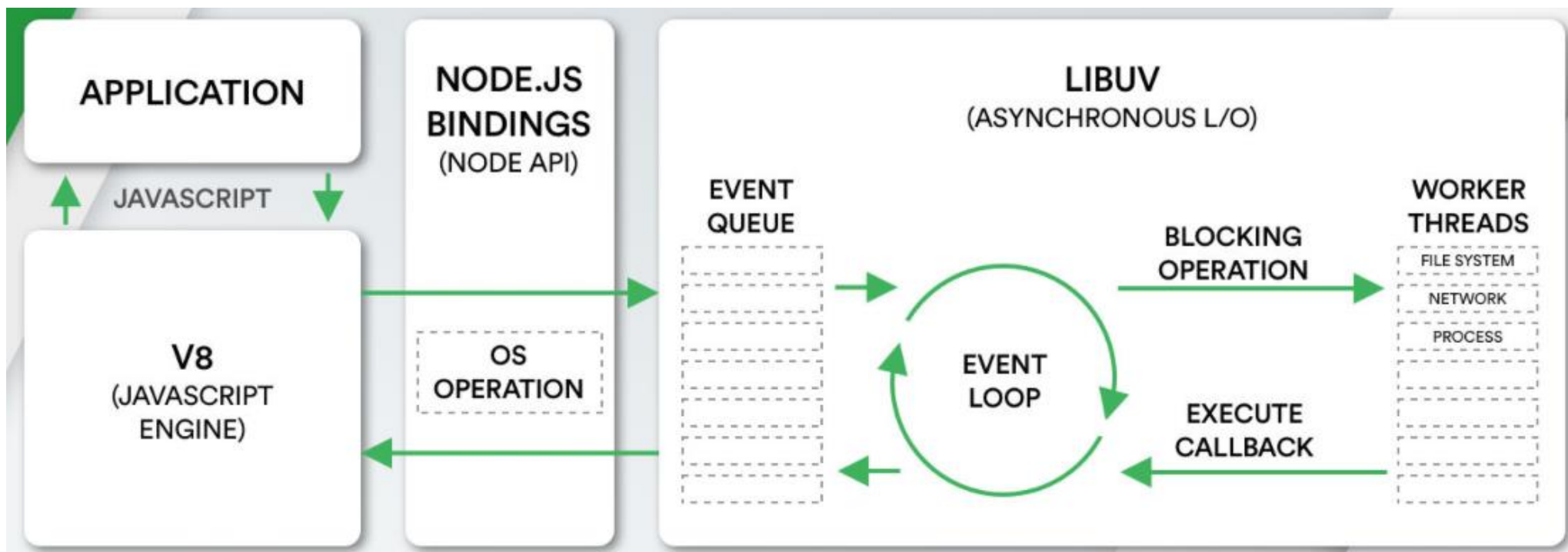
- 我们可以简单理解规划出Node.js和浏览器的差异:



Node.js架构

■ 我们来看一个单独的Node.js的架构图：

- 我们编写的JavaScript代码会经过V8引擎，再通过Node.js的Bindings，将任务放到Libuv的事件循环中；
- **libuv**（Unicorn Velociraptor—独角伶盗龙）是**使用C语言编写**的库；
- libuv提供了**事件循环、文件系统读写、网络IO、线程池**等等内容；



- fs是File System的缩写，表示文件系统。
- 对于任何一个为服务器端服务的语言或者框架通常都会有自己的文件系统：
 - 因为服务器需要将各种数据、文件等放置到不同的地方；
 - 比如用户数据可能大多数是放到数据库中的（后面我们也会学习）；
 - 比如某些配置文件或者用户资源（图片、音视频）都是以文件的形式存在于操作系统上的；
- Node也有自己的文件系统操作模块，就是fs：
 - 借助于Node帮我们封装的文件系统，我们可以在任何的操作系统（window、Mac OS、Linux）上面直接去操作文件；
 - 这也是Node可以开发服务器的一大原因，也是它可以成为前端自动化脚本等热门工具的原因；

■ Node文件系统的API非常的多：

- <https://nodejs.org/docs/latest-v16.x/api/fs.html>
- 我们不可能，也没必要一个个去学习；
- 这个更多的应该是作为一个API查询的手册，等用到的时候查询即可；
- 学习阶段我们只需要学习最常用的即可；

■ 但是这些API大多数都提供三种操作方式：

- **方式一**：同步操作文件：代码会被阻塞，不会继续执行；
- **方式二**：异步回调函数操作文件：代码不会被阻塞，需要传入回调函数，当获取到结果时，回调函数被执行；
- **方式三**：异步Promise操作文件：代码不会被阻塞，通过 `fs.promises` 调用方法操作，会返回一个Promise，可以通过`then`、`catch`进行处理；

案例：获取一个文件的状态

- 我们这里以获取一个文件的状态为例：

// 1. 同步方式读取

```
const state = fs.readFileSync('./foo.txt', { encoding: 'utf-8' })
console.log(state)
```

```
fs.readFile(
  './foo.txt',
  { encoding: 'utf-8' },
  (err, data) => {
    if (err) {
      console.log("读取发生错误:", err)
    } else {
      console.log("读取文件内容:", data)
    }
  }
)
```

// 3. 异步Promise方式

```
fs.promises
  .readFile('./foo.txt', { encoding: 'utf-8' })
  .then(res => {
    console.log("结果:", res)
  }).catch(err => {
    console.log("错误:", err)
  })
```

文件描述符

■ 文件描述符 (File descriptors) 是什么呢?

- 在常见的操作系统上, 对于每个进程, 内核都维护着一张当前打开着的文件和资源的表格。
- 每个打开的文件都分配了一个称为文件描述符的简单的数字标识符。
- 在系统层, 所有文件系统操作都使用这些文件描述符来标识和跟踪每个特定的文件。
- Windows 系统使用了一个虽然不同但概念上类似的机制来跟踪资源。

■ 为了简化用户的工作, Node.js 抽象出操作系统之间的特定差异, 并为所有打开的文件分配一个数字型的文件描述符。

■ fs.open() 方法用于分配新的文件描述符。

- 一旦被分配, 则文件描述符可用于从文件读取数据、向文件写入数据、或请求关于文件的信息。

```
fs.open("../foo.txt", 'r', (err, fd) => {  
  console.log(fd);  
  fs.fstat(fd, (err, state) => {  
    console.log(state);  
  })  
})
```

文件的读写

■ 如果我们对文件的内容进行操作，这个时候可以使用文件的读写：

- `fs.readFile(path[, options], callback)`：读取文件的内容；
- `fs.writeFile(file, data[, options], callback)`：在文件中写入内容；

```
const fs = require('fs');  
...  
let content = "aaaaa";  
  
fs.writeFile('../foo.txt', content, {flag: "a+"}, err => {  
  console.log(err);  
})
```

■ 在上面的代码中，你会发现有一个对象类型，这个是写入时填写的option参数：

- **flag**：写入的方式。
- **encoding**：字符的编码；

■ 我们先来看flag:

■ flag的值有很多: https://nodejs.org/dist/latest-v14.x/docs/api/fs.html#fs_file_system_flags

- w 打开文件写入, 默认值;
- w+ 打开文件进行读写 (可读可写), 如果不存在则创建文件;
- r 打开文件读取, 读取时的默认值;
- r+ 打开文件进行读写, 如果不存在那么抛出异常;
- a 打开要写入的文件, 将流放在文件末尾。如果不存在则创建文件;
- a+ 打开文件以进行读写 (可读可写), 将流放在文件末尾。如果不存在则创建文件

■ 我们再来看看编码：

- 我之前在简书上写过一篇关于字符编码的文章：<https://www.jianshu.com/p/899e749be47c>
- 目前基本用的都是UTF-8编码；

■ 文件读取：

- 如果不填写encoding，返回的结果是Buffer；

```
const fs = require('fs');  
  
fs.readFile('../foo.txt', {encoding: 'utf-8'}, (err, data) => {  
  console.log(data);  
})
```

■ 新建一个文件夹

□ 使用fs.mkdir()或fs.mkdirSync()创建一个新文件夹:

■ 获取文件夹的内容

■ 文件重命名

```
function readDirectory(dir) {  
  fs.readdir(dir, { withFileTypes: true }, (err, files) => {  
    files.forEach(item => {  
      if (item.isDirectory()) {  
        readDirectory(`${dir}/${item.name}`)  
      } else {  
        console.log(item.name)  
      }  
    })  
  })  
}  
  
readDirectory('./why')
```

```
fs.rename('./why', './coder', err => {  
  console.log(err);  
})
```

■ Node中的核心API都是基于异步事件驱动的：

- 在这个体系中，某些对象（发射器（Emitters））发出某一个事件；
- 我们可以监听这个事件（监听器 Listeners），并且传入的回调函数，这个回调函数会在监听到事件时调用；

■ 发出事件和监听事件都是通过EventEmitter类来完成的，它们都属于events对象。

- `emitter.on(eventName, listener)`：监听事件，也可以使用 `addListener`；
- `emitter.off(eventName, listener)`：移除事件监听，也可以使用 `removeListener`；
- `emitter.emit(eventName[, ...args])`：发出事件，可以携带一些参数；

```
const EventEmitter = require('events')

const emitter = new EventEmitter()

function handleWhy(name, age) {
  console.log('监听到why事件', name, age)
}

emitter.on('why', handleWhy)
emitter.off('why', handleWhy)

setTimeout(() => {
  emitter.emit('why', 'why', 18)
}, 2000);
```

■ EventEmitter的实例有一些属性，可以记录一些信息：

- ❑ `emitter.eventNames()`：返回当前 EventEmitter对象注册的事件字符串数组；
- ❑ `emitter.getMaxListeners()`：返回当前 EventEmitter对象的最大监听器数量，可以通过`setMaxListeners()`来修改，默认是10；
- ❑ `emitter.listenerCount(事件名称)`：返回当前 EventEmitter对象某一个事件名称，监听器的个数；
- ❑ `emitter.listeners(事件名称)`：返回当前 EventEmitter对象某个事件监听器上所有的监听器数组；

```
console.log(bus.eventNames());  
console.log(bus.getMaxListeners());  
console.log(bus.listenerCount("click"));  
console.log(bus.listeners("click"));
```

■ EventEmitter的实例方法补充:

- ❑ emitter.once(eventName, listener): 事件监听一次
- ❑ emitter.prependListener(): 将监听事件添加到最前面
- ❑ emitter.prependOnceListener(): 将监听事件添加到最前面, 但是只监听一次
- ❑ emitter.removeAllListeners([eventName]): 移除所有的监听器

```
emitter.once('click', (args) => {  
  console.log("a监听到事件", args);  
})
```

```
emitter.prependOnceListener("click", (args) => {  
  console.log("c监听到事件", args);  
})
```

```
emitter.prependListener("click", (args) => {  
  console.log("b监听到事件", args);  
})
```

```
emitter.removeAllListeners();  
emitter.removeAllListeners("click");
```

数据的二进制

- 计算机中所有的内容：文字、数字、图片、音频、视频最终都会使用二进制来表示。
- JavaScript可以直接去处理非常直观的数据：比如字符串，我们通常展示给用户的也是这些内容。
- 不对啊，JavaScript不是也可以处理图片吗？
 - 事实上在网页端，图片我们一直是交给浏览器来处理的；
 - JavaScript或者HTML，只是负责告诉浏览器一个图片的地址；
 - 浏览器负责获取这个图片，并且最终讲这个图片渲染出来；
- 但是对于服务器来说是不一样的：
 - 服务器要处理的本地文件类型相对较多；
 - 比如某一个保存文本的文件并不是使用 utf-8进行编码的，而是用 GBK，那么我们必须读取到他们的二进制数据，再通过GBK转换成对应的文字；
 - 比如我们需要读取的是一张图片数据（二进制），再通过某些手段对图片数据进行二次的处理（裁剪、格式转换、旋转、添加滤镜），Node中有一个Sharp的库，就是读取图片或者传入图片的Buffer对其再进行处理；
 - 比如在Node中通过TCP建立长连接，TCP传输的是字节流，我们需要将数据转成字节再进行传入，并且需要知道传输字节的大小（服务端需要根据大小来判断读取多少内容）；

Buffer和二进制

- 我们会发现，对于前端开发来说，通常**很少会和二进制直接打交道**，但是对于服务器端为了做很多的功能，我们**必须直接去操作其二进制的数据**；
- 所以Node为了可以方便开发者完成更多功能，提供给了我们一个类Buffer，并且它是全局的。
- 我们前面说过，Buffer中存储的是二进制数据，那么到底是如何存储呢？
 - 我们可以将Buffer看成是一个存储二进制的数组；
 - 这个数组中的**每一项**，可以**保存8位二进制**： 0000 0000
- 为什么是8位呢？
 - 在计算机中，**很少的情况我们会直接操作一位二进制**，因为一位二进制存储的数据是非常有限的；
 - 所以通常会**将8位合在一起作为一个单元**，这个单元称之为**一个字节（byte）**；
 - **也就是说 1byte = 8bit**，1kb=1024byte，1M=1024kb；
 - 比如很多编程语言中的**int类型是4个字节**，**long类型是8个字节**；
 - 比如**TCP传输的是字节流**，在**写入和读取时都需要说明字节的个数**；
 - 比如**RGB的值分别都是255**，所以本质上在计算机中都是用**一个字节存储的**；

Buffer和字符串

- Buffer相当于是一个字节的数组，数组中的每一项对应一个字节的大小：
- 如果我们希望将一个字符串放入到Buffer中，是怎么样的过程呢？

```
const buffer01 = new Buffer("why");  
console.log(buffer01);
```

- 它是怎样的过程呢？



```
const buffer2 = Buffer.from("why");  
console.log(buffer2);
```

如果是中文呢？

■ 默认编码：utf-8

```
const buffer3 = Buffer.from("王红元");  
console.log(buffer3); // <Buffer e7·8e·8b·e7·ba·a2·e5·85·83>  
  
const str = buffer3.toString();  
console.log(str); // 王红元
```

■ 如果编码和解码不同：

```
const buffer3 = Buffer.from("王红元", 'utf16le');  
console.log(buffer3);  
  
const str = buffer3.toString('utf8');  
console.log(str);  
,
```

Buffer的其他创建

- Class: Buffer

- Static method: `Buffer.alloc(size[, fill[, encoding]])`
- Static method: `Buffer.allocUnsafe(size)`
- Static method: `Buffer.allocUnsafeSlow(size)`
- Static method: `Buffer.byteLength(string[, encoding])`
- Static method: `Buffer.compare(buf1, buf2)`
- Static method: `Buffer.concat(list[, totalLength])`
- Static method: `Buffer.from(array)`
- Static method: `Buffer.from(arrayBuffer[, byteOffset[, length]])`
- Static method: `Buffer.from(buffer)`
- Static method: `Buffer.from(object[, offsetOrEncoding[, length]])`
- Static method: `Buffer.from(string[, encoding])`
- Static method: `Buffer.isBuffer(obj)`
- Static method: `Buffer.isEncoding(encoding)`

Buffer.alloc

■ 来看一下Buffer.alloc:

- 我们会发现创建了一个8位长度的Buffer，里面所有的数据默认为00;

```
const buffer01 = Buffer.alloc(8);  
console.log(buffer01); // <Buffer 00 00 00 00 00 00 00 00>
```

■ 我们也可以对其进行操作

```
buffer01[0] = 'w'.charCodeAt();  
buffer01[1] = 100;  
buffer01[2] = 0x66;  
console.log(buffer01);  
  
console.log(buffer01[0]);  
console.log(buffer01[0].toString(16));
```

Buffer和文件读取

■ 文本文件的读取:

```
fs.readFile('./test.txt', (err, data) => {  
  console.log(data); // <Buffer 48 65 6c 6c 6f 20 57 6f 72 6c 64>  
  console.log(data.toString()); // Hello World  
})
```

■ 图片文件的读取

```
fs.readFile('./zznh.jpg', (err, data) => {  
  console.log(data); // <Buffer ff d8 ff e0 ... 40418 more bytes>  
});
```

Buffer的创建过程

- 事实上我们创建Buffer时，并不会频繁的向操作系统申请内存，它会默认先申请一个8 * 1024个字节大小的内存，也就是8kb

```
Buffer.poolSize = 8 * 1024;
let poolSize, poolOffset, allocPool;

const encodingsMap = ObjectCreate(null);
for (let i = 0; i < encodings.length; ++i)
  encodingsMap[encodings[i]] = i;

function createPool() {
  poolSize = Buffer.poolSize;
  allocPool = createUnsafeBuffer(poolSize).buffer;
  markAsUntransferable(allocPool);
  poolOffset = 0;
}
createPool();
```

Buffer.from源码

■ 假如我们调用Buffer.from申请Buffer:

- 这里我们以从字符串创建为例
- node/lib/buffer.js: 290行

```
Buffer.from = function from(value, encodingOrOffset, length) {  
  if (typeof value === 'string')  
    return fromString(value, encodingOrOffset);  
  
> if (typeof value === 'object' && value !== null) { ...  
  }  
  
  throw new ERR_INVALID_ARG_TYPE(  
    'first argument',  
    ['string', 'Buffer', 'ArrayBuffer', 'Array', 'Array-like Object'],  
    value  
  );  
};
```

fromString的源码

```
function fromString(string, encoding) {  
  let ops;  
  if (typeof encoding !== 'string' || encoding.length === 0) {  
    if (string.length === 0)  
      return new FastBuffer();  
    ops = encodingOps.utf8;  
    encoding = undefined;  
  } else {  
    ops = getEncodingOps(encoding);  
    if (ops === undefined)  
      throw new ERR_UNKNOWN_ENCODING(encoding);  
    if (string.length === 0)  
      return new FastBuffer();  
  }  
  return fromStringFast(string, ops);  
}
```

fromStringFast

■ 接着我们查看fromStringFast:

- 这里做的事情是判断剩余的长度是否还足够填充这个字符串;
- 如果不够, 那么就要通过 createPool 创建新的空间;
- 如果够就直接使用, 但是之后要进行 poolOffset的偏移变化;
- node/lib/buffer.js: 428行

```
function fromStringFast(string, ops) {  
  const length = ops.byteLength(string);  
  
  // 比8kb还要长  
  // Buffer.poolSize >>> 1  
  // 8kb * 8192 个字节, 向右无符号移动一位 * 4kb  
  // 如果length 大于等于4kb的, 那么就直接通过createFromString  
  if (length >= (Buffer.poolSize >>> 1))  
    return createFromString(string, ops.encodingVal);  
  
  // 如果剩余的空间不够了  
  if (length > (poolSize - poolOffset))  
    createPool();
```

```
// 在空间内分配内存  
let b = new FastBuffer(allocPool, poolOffset, length);  
const actual = ops.write(b, string, 0, length);  
if (actual !== length) {  
  // byteLength() may overestimate. That's a rare case, though.  
  b = new FastBuffer(allocPool, poolOffset, actual);  
}  
poolOffset += actual;  
alignPool();  
return b;
```